

S2N XSS Security Report

Generated at: 2026-05-13T02:50:03.369722Z

1. Executive Summary

This report was generated from an actual S2N scan result and describes confirmed XSS findings.

Primary Finding	Cross-Site Scripting Detected
Primary Severity	HIGH
Primary Confidence	FIRM
Target URL	http://host.docker.internal:3000/#/search?q=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E
Affected Parameter	q
Primary Payload	<iframe src="javascript:alert(`xss`)">
Total Findings	1
Severity Counts	{'HIGH': 1}
Total Requests	1
Source Scanner	s2n
Source Plugin	xss

The finding is confirmed because the scanner observed unsafe payload handling through HTTP reflection or browser execution evidence.

2. All Findings Summary

The S2N scan detected the following XSS finding records. The following sections summarize each finding in a vertical detail format and then analyze the primary finding.

2.1 Finding 1

ID	xss-1
Severity	HIGH
Confidence	FIRM
Target Path	/
Parameter	q
Payload	<iframe src="javascript:alert(`xss`)">
Verification	browser_dialog_triggered
Reflected	false

3. Representative Finding Detail

ID	xss-1
Title	Cross-Site Scripting Detected
Vulnerability Type	XSS
XSS Type	dom
CWE	CWE-79
OWASP Category	Injection / Cross-Site Scripting
HTTP Method	GET
Parameter	q
Injection Context	url_parameter
Reflection Detected	true
Reflected Value	<iframe src="javascript:alert(`xss`)">
Verification	payload_reflected
Risk Score	85
Likelihood	High
Impact	High

Description

User-controlled input was reflected or executed in an unsafe browser/HTML context. This may allow JavaScript execution in a victim browser.

4. Evidence

Raw Evidence

```
XSS evidence confirmed by scanner; payload=<iframe src="javascript:alert(`xss`)">;  
verification=browser_dialog_triggered; status_code=200; method=GET; parameter=q;  
discovery_mode=browser_dom_probe;
```

Response Snippet

```
XSS evidence confirmed by scanner; payload=<iframe src="javascript:alert(`xss`)">;  
verification=browser_dialog_triggered; status_code=200; method=GET; parameter=q;  
discovery_mode=browser_dom_probe;
```

5. XSSAgent Judgement

Task	confirm_xss
Should Run	true
Context Known	true
Selected Plugin	-
Severity	HIGH
Confidence	100
Fallback	verify_xss_aggressive
Reason	XSS evidence confirmed by scanner.

6. Risk & Remediation

Risk

Severity	HIGH
Risk Score	85
Likelihood	High
Impact	High
Business Impact	An attacker may execute arbitrary JavaScript in a victim browser, steal session data, perform actions as the user, or manipulate page content.

Scoring Basis

- Severity is high because browser-executable JavaScript was confirmed by scanner evidence.
- Confidence is firm because the payload and execution/reflection evidence were verified by the scanner.
- Likelihood is high when user-controlled input is returned without context-aware output encoding.
- Impact is high because reflected script execution can lead to session theft, user action abuse, or page manipulation.

Recommended Remediation

1. Apply context-aware output encoding before inserting user-controlled input into HTML.
2. Validate and constrain user input on the server side.
3. Avoid directly reflecting raw request parameters into responses.
4. Use safe DOM APIs such as `textContent` instead of `innerHTML` where possible.
5. Add Content Security Policy as a defense-in-depth control.
6. Re-test the affected parameter with the same payload after remediation.

7. Internal Knowledge Retrieved by ChromaDB

7.1 Recommended Fix Summary

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.9306

Recommended Fix

- Apply context-aware output encoding.
- Escape HTML body output using HTML entity encoding.
- Escape HTML attribute values.
- Avoid inserting user input directly into JavaScript context.
- Prefer safe DOM APIs such as `textContent` instead of `innerHTML`.
- Validate and normalize input on the server side.
- Use Content Security Policy as a defense-in-depth control.

7.2 HTML Body Context Guidance

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.9229

HTML Body Context

When untrusted input is rendered inside the HTML body, apply HTML entity encoding before rendering the value. For example, `<script>alert(1)</script>` should be rendered as `<script>alert(1)</script>`.

7.3 XSS Remediation Guide

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.8591

HTML Attribute Context

When untrusted input is rendered inside an HTML attribute, apply attribute encoding and quote escaping. Attribute context requires careful handling of quotes, spaces, and event handler injection.

8. Official Reference Mapping

8.1 OWASP - OWASP XSS Prevention Cheat Sheet

Source	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
Source Mode	official_catalog
Score	23.7

OWASP recommends context-aware output encoding and layered defensive controls to prevent Cross-Site Scripting. Different output contexts such as HTML body, HTML attributes, JavaScript, CSS, and URL contexts require different encoding rules. Recommended use: Use this reference as the primary remediation guideline for XSS output encoding. Official URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html Source URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

8.2 CWE - CWE-79: Improper Neutralization of Input During Web Page Generation

Source	https://cwe.mitre.org/data/definitions/79.html
Source Mode	official_catalog
Score	20.7

CWE-79 classifies Cross-Site Scripting as improper neutralization of input during web page generation. Recommended use: Use this reference to classify the weakness and map the finding to CWE-79. Official URL: <https://cwe.mitre.org/data/definitions/79.html> Source URL: <https://cwe.mitre.org/data/definitions/79.html>

8.3 MDN - MDN Content Security Policy Guide

Source	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
Source Mode	official_catalog
Score	19.7

MDN describes Content Security Policy as a defense-in-depth mechanism that can help reduce the impact of XSS by controlling script execution sources. Recommended use: Use this reference as a defense-in-depth recommendation, not as a replacement for output encoding. Official URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP> Source URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

8.4 MDN - MDN Cross-site scripting XSS

Source	https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS
Source Mode	official_catalog
Score	18.2

MDN explains that XSS can occur when attacker-controlled input is included in a page without ensuring that it cannot execute as JavaScript. Recommended use: Use this reference to explain the browser-side impact and general mitigation direction. Official URL: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>
Source URL: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>

9. Retest Recommendation

Re-run the S2N XSS plugin against the affected endpoint after remediation.

Target URL	http://host.docker.internal:3000/#/search? q=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E
Parameter	q
Payload	<iframe src="javascript:alert(`xss`)">
Expected Secure Result	The payload should be encoded or rejected and must not execute as JavaScript.

Retest Steps

1. Send the same GET request to the affected endpoint with the original payload.
2. Verify that < and > are encoded as < and >, or that the input is rejected.
3. Open the response in a browser and confirm that no JavaScript alert or script execution occurs.
4. Re-run the S2N XSS plugin and confirm that the finding count for this endpoint is zero.